Acceptance-Testing Methodology for AI-Generated Code from Use Cases with UI/DB Integration

Acceptance-Testing Methodology for AI-Generated Code from Use Cases

Illustrated with *Museum Heritage Tour Designer* Use Case 1

Table of Contents

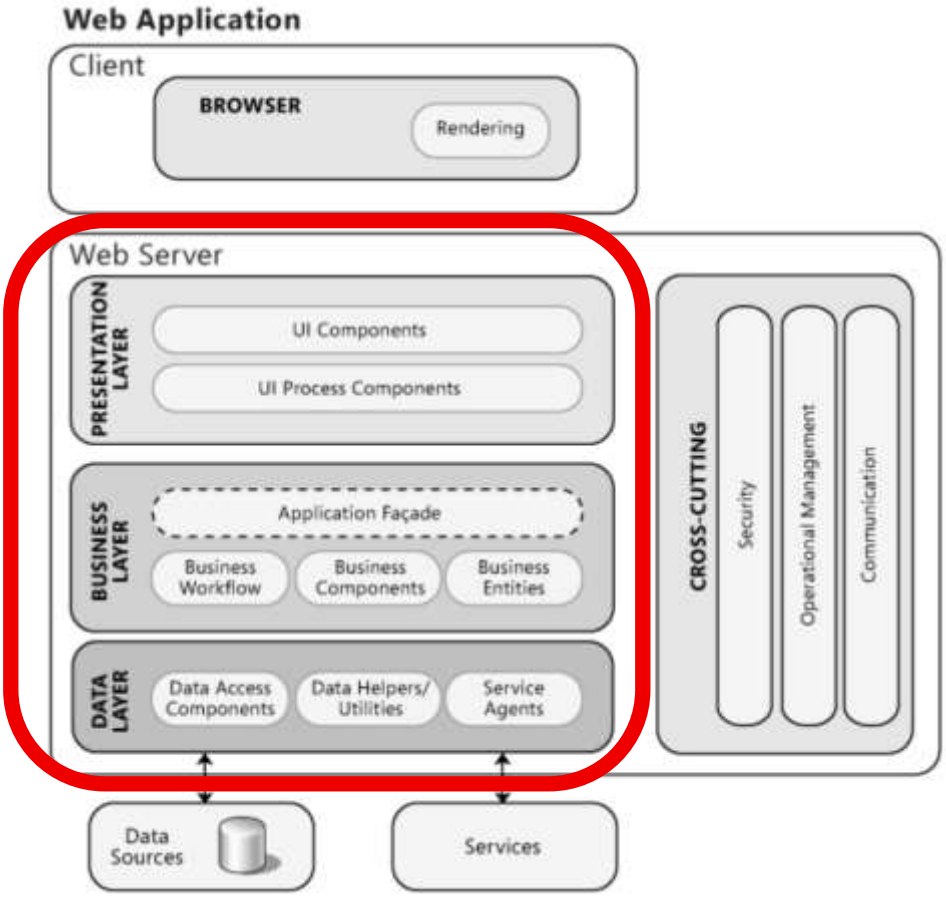
1. Purpose
2. Chosen Use Case
3. Step-by-Step Methodology
4. Worked Example Using Use Case 1
5. How to Integrate UI and Database Without Losing Acceptance-Test Rigor
6. What Can Be Automated by the AI Model
7. A Practical Prompt Pipeline
8. Recommended Artifacts
9. Conclusion

Purpose

The goal is to use **acceptance tests as the main validation contract** between a use case and the code generated by an AI model. In this approach:

- the use case is normalized into precise, testable behavior,
- that behavior is written in **Gherkin**,
- the AI model generates code to satisfy the Gherkin scenarios,
- failing scenarios become the feedback loop for repair.

Because you explicitly want the **UI and database integrated** into the plan, the methodology below keeps them in scope from the start instead of postponing them.



Chosen Use Case

I will use **Use Case 1: Generate Thematic Tour** from your uploaded document. It states that the Tour Designer selects a location and scope, specifies a target audience, the system retrieves matching exhibits/stops from the Content Database, sends them to the Narrative Generation Service, and displays a generated Draft Tour with estimated duration and stop sequence. The entry condition is that the Tour Designer is logged in and the Content Database is operational; the exit condition is that a new Draft Tour is created and saved; and the quality requirements are that at least **80%** of selected stops are linked thematically and generation time is under **5 seconds**.

Use case 1:

Use case name	Generate Thematic Tour
Participating actors	**Tour Designer, Content Database, Narrative Generation Service**
Flow of events	1. **Tour Designer** selects a museum/site location and defines the **Tour Scope** (e.g., "Greek Pottery," "Medieval Europe"). 2. **Tour Designer** specifies the **Target Audience** (e.g., "Adults," "10-12 Year Olds"). 3. System retrieves relevant **Exhibits/Stops** from the **Content Database** that match the scope. 4. System sends the selected stops and audience profile to the **Narrative Generation Service.** 5. The **Narrative Generation Service** constructs a **Thematic Narrative Flow** connecting the stops. 6. System displays the generated **Draft Tour** with an estimated duration and stop sequence to the **Tour Designer**.
Entry conditions	**Tour Designer** is logged in, and the **Content Database** is operational and populated with exhibit data.
Exit conditions	A new **Draft Tour** is created, saved, and ready for content review and localization.
Quality requirements	The generated narrative must link at least **80%** of the selected stops thematically. Tour generation time must be under **5 seconds**.

The document's later system-analysis pages also identify relevant entities and boundaries that help with UI and DB integration: Tour, Tour Stop, and Narrative appear as core entity objects, and Tour Designer UI (System) appears as the main boundary interface. The Narrative Generation Service is explicitly listed as a control object.

Step-by-Step Methodology

Step 1 — Normalize the use case into a structured requirement

Extract and rewrite the use case as:

- actor(s)

- goal
- business result
- preconditions
- main flow
- postconditions
- quality requirements
- assumptions needed for testing

This step removes ambiguity before any code is generated.

Step 2 — Convert the use case into explicit acceptance checks

Each sentence in the flow and each quality requirement must become something testable.

For example:

- “System displays the generated Draft Tour” becomes a UI-visible outcome.
- “Draft Tour is created and saved” becomes a database persistence outcome.
- “80% thematic linkage” becomes a measurable oracle.
- “under 5 seconds” becomes a timing oracle.

Step 3 — Freeze a stable acceptance boundary

Choose one stable system boundary that acceptance tests will exercise. Since you want UI and DB included, the acceptance boundary should span:

- the **actual Tour Designer UI flow** for this use case,
- the **application service/API** behind it,
- the **real persistence layer used for tests**,
- and a deterministic version of the narrative-generation dependency.

The point is not to test every internal class directly. The point is to test the end-to-end observable contract.

Step 4 — Define test oracles

A scenario is only useful if it has a clear pass/fail oracle.

For this use case, the most important oracles are:

- UI oracle: the Draft Tour preview is shown
- persistence oracle: a Draft Tour record is saved

- selection oracle: retrieved stops match the requested scope
- thematic oracle: thematicCoverage >= 0.80
- performance oracle: generationTimeMs < 5000

Step 5 — Write Gherkin feature files

Turn the acceptance checks into Gherkin scenarios covering:

- happy path
- entry/precondition failures
- edge cases
- quality requirements
- persistence outcomes

Gherkin becomes the **executable contract** for the AI-generated code.

Step 6 — Derive the minimal integrated UI and DB design

Before generating code, use the AI model to derive:

- screens/pages/views needed
- user inputs and outputs
- route/navigation or screen transitions
- entities/tables/records needed
- seed/reset data for tests

Because UI and DB are in scope, this step should happen **before** implementation generation.

Step 7 — Generate the acceptance harness

Have the AI model generate a minimal in-repo harness that can:

- seed/reset the test database
- simulate user actions through the UI
- trigger the use-case flow
- inspect visible output
- inspect saved DB state
- record timing
- produce clear failure bundles

This keeps tooling light and centered around project artifacts.

Step 8 — Ask the AI model to generate the implementation

Provide the AI with:

- the use case
- the Gherkin file
- the UI/DB design
- the accepted oracles
- any runtime constraints

Tell it to implement only what is needed to make the scenarios pass.

Step 9 — Run acceptance tests and collect a failure bundle

When scenarios fail, collect:

- failing scenario ID
- expected vs actual
- logs
- UI symptom
- DB state
- timing values
- relevant code pointers

This structured failure bundle is the input to the repair loop.

Step 10 — Ask the AI model for a minimal patch

Use the failure bundle to request:

- likely root cause
- minimal patch plan or diff
- verification notes
- rerun scope

Repeat until the acceptance suite is green.

Step 11 — Turn discovered bugs into regression scenarios

If a meaningful defect is found, add a new scenario or strengthen an existing one. The acceptance suite then becomes a growing regression contract.

Worked Example Using Use Case 1

1) Structured requirement

Use case
Generate Thematic Tour

Actors
Tour Designer, Content Database, Narrative Generation Service.

Goal
Create a draft thematic tour for a selected location, scope, and audience.

Preconditions

- Tour Designer is logged in
- Content Database is operational and populated.

Main result

- Draft Tour is displayed and saved.

Quality constraints

- thematic linkage at least 80%
- generation time under 5 seconds.

2) Acceptance checks

Use-case statement	Acceptance check
Tour Designer selects location and scope	UI provides location and scope inputs and captures values
Tour Designer specifies target audience	UI provides audience selector/input
System retrieves relevant exhibits/stops	Retrieved stop set is filtered by scope
Narrative Generation Service constructs thematic flow	Result includes thematic narrative data or a measurable linkage score
System displays Draft Tour with duration and stop sequence	UI shows ordered stops and estimated duration
Draft Tour is created and saved	DB contains a saved Draft Tour record after success
80% linkage	thematicCoverage >= 0.80
under 5 seconds	generationTimeMs < 5000

3) Example Gherkin

Feature: Generate Thematic Tour

Background:

Given the Tour Designer is logged in

And the Content Database is operational

And the database contains exhibits for location "Museum-A"

And the Narrative Generation Service is available

Scenario: UC1-S01 Generate and save a thematic draft tour

When I choose location "Museum-A"

And I choose scope "Greek Pottery"

And I choose audience "Adults"

And I request thematic tour generation

Then a Draft Tour is displayed

And the Draft Tour shows an ordered stop sequence

And the Draft Tour shows an estimated duration

And the Draft Tour is saved

Scenario: UC1-S02 Generated tour meets thematic linkage threshold

When I choose location "Museum-A"

And I choose scope "Greek Pottery"

And I choose audience "Adults"

And I request thematic tour generation

Then the thematic coverage is at least 0.80

Scenario: UC1-S03 Generation completes within performance limit

When I choose location "Museum-A"

And I choose scope "Greek Pottery"

And I choose audience "Adults"

And I request thematic tour generation

Then the generation time is under 5000 milliseconds

Scenario: UC1-S04 Database unavailable blocks generation

Given the Content Database is unavailable

When I choose location "Museum-A"

And I choose scope "Greek Pottery"

And I choose audience "Adults"

And I request thematic tour generation

Then I see an error indicating content data is unavailable

And no Draft Tour is saved

4) Minimal integrated UI design for this use case

Using the document's Tour Designer UI (System) boundary object as the main interface, the minimal UI for this use case should include: location selector, scope selector, audience selector, generate button, result panel for the Draft Tour, stop sequence display, estimated duration display, and an error area.

5) Minimal integrated DB design for this use case

Using the entity objects listed in the document, the minimum DB model for this use case should center on:

- Tour
- Tour Stop
- Narrative

A practical test-time schema would include:

- Exhibit or StopCatalog table for content lookup
- DraftTour table
- DraftTourStop table
- Narrative or GeneratedNarrative table

The goal is not to design the whole platform first. The goal is to support exactly what UC1 needs to execute and be validated.

6) Acceptance harness for this use case

The AI-generated harness should be able to:

- reset the DB to a known state
- seed exhibits for "Museum-A" and "Greek Pottery"
- simulate Tour Designer selections in the UI
- trigger generation
- read visible results
- verify DB side effects
- measure timing
- record thematic coverage

How to Integrate UI and Database Without Losing Acceptance-Test Rigor

Your constraint is important: integrating UI and DB early often makes tests fragile. The way to keep rigor is to use these rules.

Rule 1 — Keep the acceptance contract stable

Do not let UI details redefine the business contract. The Gherkin should still be about observable behavior, not widget internals.

Rule 2 — Seed deterministic DB data for every scenario

Before each scenario:

- clear existing Draft Tours
- load a known exhibit dataset
- load fixed scope and audience fixtures

That makes acceptance tests repeatable.

Rule 3 — Treat external AI/service dependencies as deterministic test doubles

For the Narrative Generation Service, keep a deterministic test implementation during acceptance runs. The document treats this service as a control object, which makes it a good place for a controlled adapter in tests.

Rule 4 — Use the real UI flow, but keep it thin

The UI should collect inputs, trigger generation, and render outputs. It should not contain hidden business logic that makes acceptance behavior hard to verify.

Rule 5 — Put measurable quality outputs into the response or persisted result

If the use case says 80% linkage and under 5 seconds, expose those values explicitly so tests can assert them.

What Can Be Automated by the AI Model

A large part of the process can be automated by the AI model itself.

Stage	What the AI can generate
Use case analysis	structured requirement summary
Acceptance design	Gherkin feature file
UI design	fields, views, flows, screen states
DB design	minimal schema, fixtures, seed/reset rules
Harness generation	step definitions, scenario runner, DB reset helpers
Implementation	UI, service, DB logic, adapters
Repair loop	failure diagnosis and minimal patch

Example automation process

1. **Use case → structured requirement**
   Prompt the AI to extract actors, preconditions, flows, postconditions, and NFRs.
2. **Structured requirement → Gherkin**
   Prompt the AI to convert acceptance criteria into scenario IDs and feature files.
3. **Gherkin → UI/DB design**
   Prompt the AI to infer screens, routes, DB entities, persistence events, and test fixtures.
4. **Gherkin + design → implementation**
   Prompt the AI to generate minimal integrated code.
5. **Failure bundle → patch**
   Prompt the AI to diagnose failing scenarios and suggest the smallest safe repair.

A Practical Prompt Pipeline

A compact pipeline that avoids heavy external tooling is:

1. **Prompt 1 — Use Case → Structured Requirement + Gherkin**
2. **Prompt 2 — Gherkin → UI + DB + Harness Design**
3. **Prompt 3 — Design + Gherkin → Implementation**
4. **Run acceptance tests**

5. **Prompt 4 — Failure Bundle → Minimal Patch**
6. **Repeat until green**

This works especially well for the Museum Heritage Tour Designer because the document already gives you use cases plus system-analysis hints about entities, boundaries, and control objects.

Recommended Artifacts

For each use case, keep these artifacts:

- original use case text
- structured requirement
- Gherkin feature file
- UI design note
- DB/persistence design note
- acceptance harness spec
- generated implementation
- failure bundles
- regression scenarios

For UC1 specifically, the most important artifacts are:

- UC1_Generate_Thematic_Tour.feature
- UC1_UI_Design.md
- UC1_DB_Design.md
- UC1_Test_Fixtures.md
- UC1_Failure_Bundles/

Conclusion

A strong acceptance-testing methodology for AI-generated software should treat Gherkin as the **main behavioral contract** between requirements and code. Using the *Generate Thematic Tour* use case, you can build a disciplined loop:

use case → acceptance checks → Gherkin → integrated UI/DB design → generated implementation → acceptance run → failure bundle → patch

Because you want UI and database included, the right compromise is not to avoid them. It is to integrate them early while controlling them with:

- stable acceptance boundaries,
- deterministic test data,
- measurable oracles,
- and AI-driven repair loops.

That gives you an approach that is both practical for AI-assisted development and rigorous enough for validating generated code against the actual requirements in the use case document.

Minimal Prompt Templates for Acceptance-Testing the Use Case Generate Thematic Tour

Table of Contents

1. Overview
2. Minimal Prompt Set
3. Template 1 — Use Case to Structured Requirement and Gherkin
4. Template 2 — Gherkin to Integrated UI, Database, and Acceptance-Harness Design
5. Template 3 — Contract and Design to Integrated Implementation
6. Template 4 — Failure Bundle to Minimal Patch
7. Why These Outputs Matter
8. Recommended Usage Pattern

Overview

For **Generate Thematic Tour**, the source use case says the Tour Designer selects a location and tour scope, specifies a target audience, the system retrieves matching exhibits/stops from the Content Database, sends them to the Narrative Generation Service, and displays a generated Draft Tour with estimated duration and stop sequence. The entry conditions require the Tour Designer to be logged in and the Content Database to be operational, the exit condition is that a new Draft Tour is created and saved, and the quality requirements are at least **80% thematic linkage** and **under 5 seconds** generation time.

The same document's system analysis identifies `Tour`, `Tour Stop`, and `Narrative` as core entities, `Tour Designer UI (System)` as the main boundary object, and `Narrative Generation Service` as a control object. Those are useful anchors for integrating UI and database design into the acceptance-testing flow.

A **minimal** prompt set for this workflow is:

1. **Use Case → Structured Requirement + Gherkin**
2. **Gherkin → Integrated UI + DB + Acceptance Harness Design**
3. **Contract + Design → Integrated Implementation**
4. **Failure Bundle → Minimal Patch**

That is enough to cover the full loop from requirement to repair, without adding extra prompt layers.

Minimal Prompt Set

Template	Purpose	When to use it
T1	Turn the use case into a precise acceptance contract	First
T2	Derive the minimal UI, DB, and harness needed to validate that contract	After T1
T3	Generate the integrated implementation	After T2
T4	Repair failures without weakening the contract	Whenever tests fail

---

Template 1 — Use Case to Structured Requirement and Gherkin

Expanded template

```
You are an expert software analyst and acceptance-test designer.

GOAL
Convert the provided use case into:
1) a structured requirement specification
2) explicit acceptance checks
3) a Gherkin feature file that can validate AI-generated code

CONTEXT
- The implementation will include UI and database behavior
- Acceptance tests must remain rigorous and scenario-driven
- Avoid inventing hidden requirements, label assumptions explicitly

INPUT
- Use case text: <<PASTE USE CASE>>
- Optional system-analysis hints: <<PASTE ENTITY/BOUNDARY/CONTROL NOTES OR NONE>>

INSTRUCTIONS
1) Restate the use case in structured form:
   - Use case name
   - Participating actors
   - Goal
   - Preconditions
   - Main success flow
   - Success postconditions
   - Failure postconditions
   - Quality requirements
   - Assumptions (A1, A2, ...)
2) Convert each flow step and each quality requirement into explicit acceptance checks.
3) Define the key acceptance oracles:
   - UI oracles
   - database/persistence oracles
   - quality/performance oracles
4) Write a Gherkin feature file with:
   - Background
   - happy-path scenario(s)
   - negative/precondition scenario(s)
   - persistence scenario(s)
   - quality/NFR scenario(s)
5) Use stable scenario IDs in the format:
   - UC1-S01, UC1-S02, ...
6) Keep the scenarios implementation-neutral but observable.

OUTPUT FORMAT
A) Structured Requirement
B) Acceptance Checks Table
C) Acceptance Oracles
```

D) Gherkin Feature File

Applied prompt for Generate Thematic Tour

```
You are an expert software analyst and acceptance-test designer.

GOAL
Convert the provided use case into:
1) a structured requirement specification
2) explicit acceptance checks
3) a Gherkin feature file that can validate AI-generated code

CONTEXT
- The implementation will include UI and database behavior
- Acceptance tests must remain rigorous and scenario-driven
- Avoid inventing hidden requirements; label assumptions explicitly

INPUT
- Use case text:
Use case name: Generate Thematic Tour
Participating actors: Tour Designer, Content Database, Narrative Generation Service
Flow of events:
1. Tour Designer selects a museum/site location and defines the Tour Scope.
2. Tour Designer specifies the Target Audience.
3. System retrieves relevant Exhibits/Stops from the Content Database that match the
scope.
4. System sends the selected stops and audience profile to the Narrative Generation
Service.
5. The Narrative Generation Service constructs a Thematic Narrative Flow connecting the
stops.
6. System displays the generated Draft Tour with an estimated duration and stop sequence
to the Tour Designer.
Entry conditions: Tour Designer is logged in, and the Content Database is operational and
populated with exhibit data.
Exit conditions: A new Draft Tour is created, saved, and ready for content review and
localization.
Quality requirements: The generated narrative must link at least 80% of the selected
stops thematically. Tour generation time must be under 5 seconds.

- Optional system-analysis hints:
Entity objects include Tour, Tour Stop, and Narrative.
Boundary object includes Tour Designer UI (System).
Control object includes Narrative Generation Service.
```

Expected output from an AI model

A) Structured Requirement

Use case name
Generate Thematic Tour

Actors
Tour Designer, Content Database, Narrative Generation Service

Goal
Generate and save a draft thematic tour for a selected location, scope, and target audience.

Preconditions

- Tour Designer is logged in
- Content Database is operational
- Exhibit data exists for the selected location/scope

Main success flow

1. Tour Designer selects location and scope
2. Tour Designer selects target audience
3. System retrieves matching stops
4. System sends selected stops and audience profile to Narrative Generation Service
5. Narrative Generation Service constructs thematic flow
6. System displays Draft Tour with stop sequence and estimated duration
7. System saves the Draft Tour

Success postconditions

- Draft Tour is displayed
- Draft Tour is saved
- Draft Tour is ready for content review and localization

Failure postconditions

- No Draft Tour is saved
- UI shows an appropriate failure message
- User remains in the generation workflow

Quality requirements

- `thematicCoverage >= 0.80`
- `generationTimeMs < 5000`

Assumptions

- **A1:** The system exposes `thematicCoverage` as a measurable value for testing.
- **A2:** Generation time is measured end-to-end at the use-case boundary.
- **A3:** If no stops match the scope, the request fails rather than producing an empty draft.

B) Acceptance Checks Table

Use-case statement	Acceptance check
Tour Designer selects location and scope	UI captures location and scope inputs
Tour Designer specifies audience	UI captures audience input
System retrieves matching stops	Returned stop set matches requested scope
Narrative Generation Service constructs thematic flow	Draft result includes thematic linkage output
System displays Draft Tour	UI shows ordered stop sequence and estimated duration
Draft Tour is created and saved	DB contains saved Draft Tour after success
80% thematic linkage	`thematicCoverage >= 0.80`
under 5 seconds	`generationTimeMs < 5000`

C) Acceptance Oracles

- **UI oracle:** Draft Tour preview is visible
- **Sequence oracle:** ordered stop list is visible
- **Duration oracle:** estimated duration is visible
- **Persistence oracle:** Draft Tour row/entity exists after success
- **Selection oracle:** selected stops match requested scope
- **Thematic oracle:** thematic coverage is at least 0.80
- **Performance oracle:** generation finishes in under 5000 ms

D) Gherkin Feature File

```
Feature: Generate Thematic Tour

  Background:
    Given the Tour Designer is logged in
    And the Content Database is operational
    And the exhibit catalog contains data for location "Museum-A"
    And the Narrative Generation Service is available

  Scenario: UC1-S01 Generate and save a thematic draft tour
    When I select location "Museum-A"
    And I select scope "Greek Pottery"
    And I select audience "Adults"
    And I request thematic tour generation
    Then a Draft Tour is displayed
    And the Draft Tour shows an ordered stop sequence
    And the Draft Tour shows an estimated duration
    And the Draft Tour is saved

  Scenario: UC1-S02 Generated tour meets thematic linkage threshold
    When I select location "Museum-A"
    And I select scope "Greek Pottery"
    And I select audience "Adults"
    And I request thematic tour generation
    Then the thematic coverage is at least 0.80

  Scenario: UC1-S03 Generation completes within time limit
    When I select location "Museum-A"
    And I select scope "Greek Pottery"
    And I select audience "Adults"
    And I request thematic tour generation
    Then the generation time is under 5000 milliseconds

  Scenario: UC1-S04 Content database unavailable
    Given the Content Database is unavailable
    When I select location "Museum-A"
    And I select scope "Greek Pottery"
    And I select audience "Adults"
    And I request thematic tour generation
    Then I see a content-data-unavailable error
    And no Draft Tour is saved
```

Explanation of the expected output

This output gives you the **acceptance contract**. It takes the source use case and turns it into a form the AI can implement against directly. The main value is that it separates:

- what the user does,

- what the system must show,
- what the system must save,
- and what quality thresholds must be met.

Without this step, later prompts tend to blur UI behavior, data persistence, and quality requirements.

Template 2 — Gherkin to Integrated UI, Database, and Acceptance-Harness Design

Expanded template

```
You are an expert software architect and acceptance-harness designer.

GOAL
Derive the minimal integrated design needed to implement and validate the use case
described by the Gherkin contract.

CONTEXT
- UI and database are in scope
- Keep the design minimal and directly traceable to the scenarios
- Avoid over-designing the full system

INPUT
- Structured requirement: <<PASTE>>
- Gherkin feature file: <<PASTE>>
- Optional system-analysis hints: <<PASTE OR NONE>>

INSTRUCTIONS
1) Identify the minimum UI needed:
   - screen/page names
   - inputs
   - outputs/results
   - errors/messages
2) Identify the minimum database/persistence design:
   - entities/tables
   - key fields
   - seed data
   - reset rules
3) Identify the minimal service/control design:
   - application service(s)
   - external/test-double dependency behavior
4) Define the acceptance harness:
   - what is seeded before each scenario
   - what is reset
   - which UI actions are simulated
   - which DB state is asserted
   - which metrics are recorded
5) Provide a traceability table:
   Scenario ID → UI elements → DB elements → service elements

OUTPUT FORMAT
A) UI Design Summary
B) Database Design Summary
C) Service/Control Design Summary
D) Acceptance Harness Design
E) Traceability Table
```

Applied prompt for Generate Thematic Tour

```
You are an expert software architect and acceptance-harness designer.

GOAL
Derive the minimal integrated design needed to implement and validate the use case
described by the Gherkin contract.

CONTEXT
- UI and database are in scope
- Keep the design minimal and directly traceable to the scenarios
- Avoid over-designing the full system

INPUT
- Structured requirement: <<output of Template 1>>
- Gherkin feature file: <<output of Template 1>>
- Optional system-analysis hints:
Tour, Tour Stop, and Narrative are core entities.
Tour Designer UI (System) is the main boundary.
Narrative Generation Service is a control object.
```

Expected output from an AI model

A) UI Design Summary

Main page/view

- `GenerateThematicTourView`

Inputs

- location selector
- scope selector
- audience selector
- generate button

Outputs

- Draft Tour panel
- ordered stop sequence
- estimated duration
- thematic coverage indicator
- error message area

B) Database Design Summary

Core entities/tables

- `ExhibitCatalog`
- `DraftTour`
- `DraftTourStop`
- `GeneratedNarrative`

Key fields

- `DraftTour`: id, location, scope, audience, estimatedDuration, thematicCoverage, createdAt
- `DraftTourStop`: draftTourId, stopId, sequenceIndex
- `GeneratedNarrative`: draftTourId, narrativeText, thematicCoverage

Seed data

- exhibits for `"Museum-A"` tagged with `"Greek Pottery"`
- one contrasting non-matching dataset for error/edge tests

Reset rules

- clear DraftTour, DraftTourStop, GeneratedNarrative before each scenario
- reseed ExhibitCatalog before each scenario

C) Service/Control Design Summary

Application service

- `GenerateThematicTourService`

External/test-double behavior

- deterministic `NarrativeGenerationService` test double returns known thematic coverage and ordered stop set during acceptance runs

D) Acceptance Harness Design

Before each scenario

- seed exhibit catalog
- reset draft-tour tables
- initialize UI in logged-in state unless scenario overrides it

Simulated actions

- choose location
- choose scope
- choose audience
- click/tap generate

Assertions

- visible Draft Tour panel
- visible stop order
- visible duration
- persisted Draft Tour
- recorded thematicCoverage
- recorded generationTimeMs
- appropriate error when DB unavailable

E) Traceability Table

Scenario ID	UI elements	DB elements	Service/control elements
UC1-S01	location/scope/audience inputs, generate button, draft tour panel	DraftTour, DraftTourStop	GenerateThematicTourService, NarrativeGenerationService
UC1-S02	thematic coverage display	GeneratedNarrative, DraftTour	NarrativeGenerationService
UC1-S03	none beyond normal results	timing record in DraftTour or response	GenerateThematicTourService
UC1-S04	error message area	no DraftTour rows created	Content DB adapter, GenerateThematicTourService

Explanation of the expected output

This output gives you the **minimum integrated design** needed to implement and validate the use case. It matters because it prevents the AI from generating too much architecture too early. It keeps UI, DB, and service boundaries tightly tied to the actual scenarios.

Template 3 — Contract and Design to Integrated Implementation

Expanded template

```
You are a senior software engineer implementing code from an acceptance contract.

GOAL
Generate the minimal integrated implementation required to make the Gherkin scenarios
pass.

CONTEXT
- UI and database are included
- Acceptance tests are the primary source of truth
- Keep the implementation modular and easy to repair
- Do not change the Gherkin unless it is contradictory

INPUT
- Structured requirement: <<PASTE>>
- Gherkin feature file: <<PASTE>>
- Integrated UI/DB/service design: <<PASTE>>
- Runtime constraints: <<PASTE language/framework/database>>

INSTRUCTIONS
1) Produce a brief implementation plan.
2) Propose a minimal file/module structure.
3) Identify the main modules/components to generate.
4) Identify acceptance-relevant state and behaviors.
5) Keep the implementation small and scenario-focused.
6) Include minimal run/test instructions.

OUTPUT FORMAT
A) Implementation Plan
B) Proposed File/Module Structure
C) Main Components to Generate
D) Acceptance-Relevant State and Behaviors
E) Minimal Run/Test Instructions
```

Applied prompt for Generate Thematic Tour

```
You are a senior software engineer implementing code from an acceptance contract.

GOAL
Generate the minimal integrated implementation required to make the Gherkin scenarios
pass.

CONTEXT
- UI and database are included
- Acceptance tests are the primary source of truth
- Keep the implementation modular and easy to repair
- Do not change the Gherkin unless it is contradictory

INPUT
- Structured requirement: <<output of Template 1>>
- Gherkin feature file: <<output of Template 1>>
- Integrated UI/DB/service design: <<output of Template 2>>
- Runtime constraints:
  - web application
  - Tour Designer UI
  - database-backed persistence
  - deterministic narrative-generation adapter for acceptance testing
```

Expected output from an AI model

A) Implementation Plan

1. Create the Tour Designer UI page for thematic tour generation
2. Add form inputs for location, scope, and audience
3. Add Generate action handler
4. Implement `GenerateThematicTourService`
5. Implement database persistence for Draft Tour, Draft Tour Stops, and Generated Narrative
6. Add deterministic Narrative Generation adapter for acceptance tests
7. Render Draft Tour preview
8. Persist results after successful generation
9. Surface generation errors to the UI
10. Capture thematic coverage and generation time for assertions

B) Proposed File/Module Structure

```
src/
  ui/
    GenerateThematicTourPage
    DraftTourPreview
  application/
    GenerateThematicTourService
  domain/
    Tour
    TourStop
    Narrative
  infrastructure/
    ExhibitRepository
    DraftTourRepository
    NarrativeGenerationAdapter
  testsupport/
    FixtureSeeder
    DatabaseReset
```

```
AcceptanceHooks
```

C) Main Components to Generate

- `GenerateThematicTourPage`
- `DraftTourPreview`
- `GenerateThematicTourService`
- `ExhibitRepository`
- `DraftTourRepository`
- `NarrativeGenerationAdapter`
- fixture/reset helpers for acceptance runs

D) Acceptance-Relevant State and Behaviors

State

- selected location
- selected scope
- selected audience
- generated draft tour
- stop sequence
- estimated duration
- thematic coverage
- generation time
- error state

Behaviors

- successful generation saves and displays Draft Tour
- DB unavailable prevents save and shows error
- thematic coverage is recorded
- generation time is recorded

E) Minimal Run/Test Instructions

- seed DB with exhibit catalog
- launch Tour Designer generation page
- run acceptance scenarios
- inspect UI and persistence results
- rerun after repair if needed

Explanation of the expected output

This output is not the full code yet. It is the **implementation blueprint** the AI should follow. That matters because it keeps the generated code tied to the contract and gives you a review checkpoint before the AI produces too much framework-specific detail.

Template 4 — Failure Bundle to Minimal Patch

Expanded template

```
You are a debugging engineer for AI-generated software.

GOAL
Repair failing acceptance scenarios with the smallest safe change.

INPUT
- Use case ID/name: <<PASTE>>
- Failing scenario(s): <<PASTE>>
- Expected result: <<PASTE>>
- Actual result: <<PASTE>>
- UI symptom: <<PASTE>>
- Database state: <<PASTE>>
- Logs/stack traces: <<PASTE>>
- Relevant code or module names: <<PASTE>>

RULES
1) Do not weaken the acceptance contract.
2) Preserve observable behavior for already-passing scenarios.
3) Prefer the smallest patch that fixes the failure.
4) Explain the likely root cause plainly.
5) State what should be re-tested after the patch.

OUTPUT FORMAT
A) Failure Summary
B) Root Cause Hypothesis
C) Minimal Patch Plan
D) Verification Notes
```

Applied prompt for Generate Thematic Tour

```
You are a debugging engineer for AI-generated software.

GOAL
Repair failing acceptance scenarios with the smallest safe change.

INPUT
- Use case ID/name: UC1 Generate Thematic Tour
- Failing scenario(s): UC1-S01 Generate and save a thematic draft tour
- Expected result: Draft Tour is displayed and saved
- Actual result: Draft Tour is displayed, but no Draft Tour record is saved
- UI symptom: Draft Tour preview appears correctly in the Tour Designer UI
- Database state: DraftTour table remains empty after successful generation
- Logs/stack traces: service returns success response, but repository save() is never
called
- Relevant code or module names:
  - GenerateThematicTourService
  - DraftTourRepository
  - GenerateThematicTourPage
```

Expected output from an AI model

A) Failure Summary

- Failing scenario: `UC1-S01`
- UI behavior succeeds: Draft Tour preview is visible
- Persistence behavior fails: Draft Tour is not saved
- The issue is limited to the save side effect

B) Root Cause Hypothesis

The generation flow builds and returns the Draft Tour view model, but the service path does not invoke the persistence step before returning success. The UI is therefore rendering transient data rather than data that has been committed to the database.

C) Minimal Patch Plan

1. Update `GenerateThematicTourService` to call `DraftTourRepository.save(...)` after successful generation
2. Ensure saved Draft Tour ID is propagated to the UI result model
3. Leave UI rendering logic unchanged
4. Leave thematic coverage and timing logic unchanged

D) Verification Notes

Re-test:

- `UC1-S01` because it directly checks save behavior
- `UC1-S02` to confirm thematic coverage still records correctly
- `UC1-S03` to confirm persistence step did not break performance budget

Explanation of the expected output

This output creates the **repair loop**. It matters because it keeps failure analysis disciplined. Instead of asking the AI to rewrite everything, you constrain it to diagnose one failing behavior, propose a small fix, and specify which scenarios to rerun.

Why These Outputs Matter

Template 1 gives you the behavioral contract.
Template 2 gives you the smallest integrated design that can satisfy that contract.
Template 3 gives you an implementation blueprint aligned with the contract.
Template 4 gives you a controlled repair loop.

Together, they keep the AI focused on:

- the use case,
- the acceptance scenarios,
- the integrated UI and DB behavior,
- and the smallest possible correction when something fails.

Recommended Usage Pattern

For **Generate Thematic Tour**, and later for the other Museum Heritage Tour Designer use cases, use the prompts in this order:

1. Run **Template 1**
2. Review and approve the structured requirement and Gherkin
3. Run **Template 2**
4. Review the UI/DB/harness design
5. Run **Template 3**
6. Generate the integrated implementation
7. Run the acceptance scenarios
8. If any scenario fails, run **Template 4**
9. Repeat until the suite is green

That gives you a compact, reusable prompt system for converting the source use case into a rigorous acceptance-testing workflow grounded in the project document's use-case text and system-analysis model.